COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	10/13/2026	**DATE SUBMITTED:**	10/13/2026
NAME:	Salcedo, Lance Cristopher S.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS ON ACTIVITY NO. 10.1 **Graphs**

1. PROCEDURE OUTPUT

Code

```
#include <iostream>
// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};
// structure to store edges
struct graphEdge {
int start_ver, end_ver, weight;
};
class DiaGraph{
// insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
    adjNode* newNode = new adjNode;
    newNode->val = value;
    newNode->cost = weight;
    newNode->next = head; // point new node to current head
    return newNode;
}
    int N; // number of nodes in the graph
    public:
    adjNode **head; //adjacency list as array of pointers
    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
    // allocate new node
    head = new adjNode*[N]();
    this->N = N;
    // initialize head pointer for all vertices
    for (int i = 0; i < N; ++i)
        head[i] = nullptr;
    // construct directed graph by adding edges to it
    for (unsigned i = 0; i < n; i++) {
```

```
        int start_ver = edges[i].start_ver;
        int end_ver = edges[i].end_ver;
        int weight = edges[i].weight;
        // insert in the beginning
        adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ve
        // point head pointer to new node
        head[start_ver] = newNode;
    }
}
    // Destructor
    ~DiaGraph() {
    for (int i = 0; i < N; i++)
        delete[] head[i];
    delete[] head;
    }
};
// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i)
{
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val
            << ", " << ptr->cost << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}
// graph implementation
int main()
{
    // graph edges array.
    graphEdge edges[] = {
    // (x, y, w) -> edge from x to y with weight w
    {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
    };
    int N = 6; // Number of vertices in the graph
    // calculate number of edges
    int n = sizeof(edges)/sizeof(edges[0]);
    // construct graph
    DiaGraph diagraph(edges, n, N);
    // print adjacency list representation of graph
    std::cout<<"Graph adjacency list "
    << "(start_ver, end_ver, weight):"<< std::endl;

    for (int i = 0; i < N; i++)
    {
    // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }
    return 0;
}
```

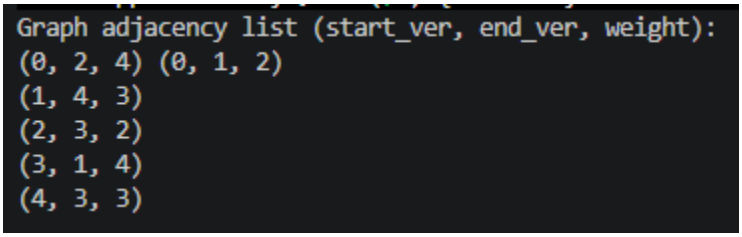
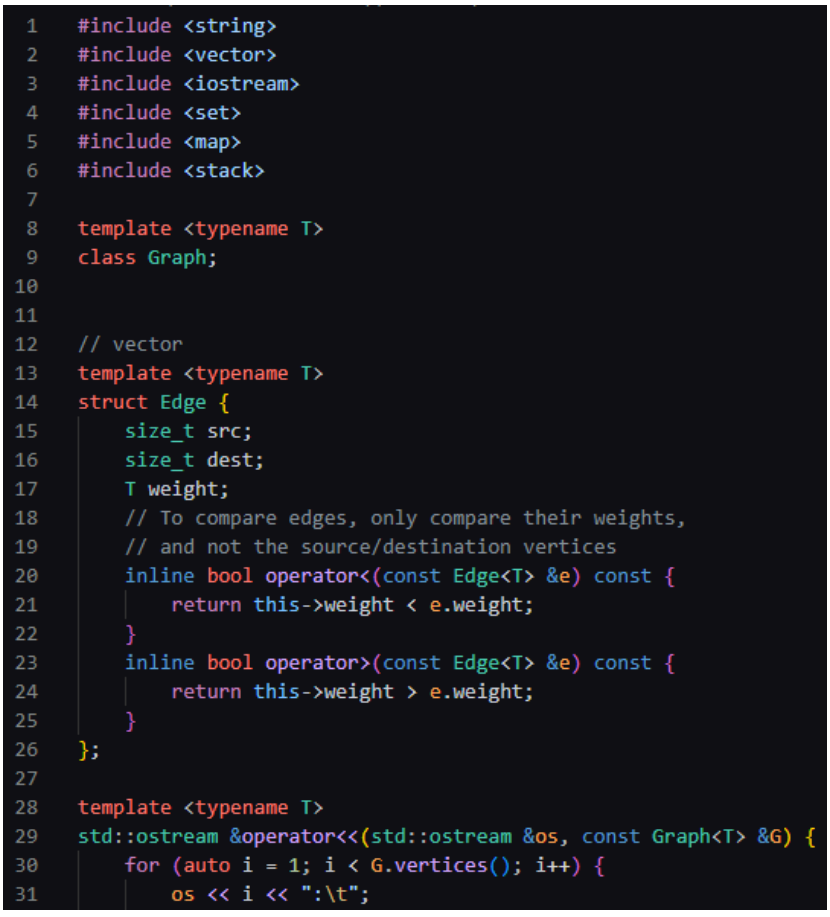
Output	
Analysis	In this algorithm, it implements a directed weighted graph. It uses adjacency-list representation. It prints each adjacent neighbors' vertices with their edge weights.'

Table 10.1 ILO A

Code	

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;


// vector
template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;
    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
    for (auto i = 1; i < G.vertices(); i++) {
        os << i << ":\t";
```

```
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

template <typename T>
class Graph {
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N) {
    }
    // Return number of vertices in the graph
    auto vertices() const {
        return V;
    }
    // Return all edges in the graph
    auto &edges() const {
        return edge_list;
    }
    void add_edge(Edge<T> &&e) {
    // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }
```

```
    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list) {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }
    // Overloads the << operator so a graph be written directly to a st
    // Can be used as std::cout << obj << std::endl;
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os,
        const Graph<U> &G);
private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};

template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest) {
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1); // Assume that DFS always starts from vertex ID 1
    while (!stack.empty()) {
        auto current_vertex = stack.top();
        stack.pop();
        // If the current vertex hasn't been visited in the past
```

```
90          if (visited.find(current_vertex) == visited.end()) {
91              visited.insert(current_vertex);
92              visit_order.push_back(current_vertex);
93              for (auto e : G.outgoing_edges(current_vertex)) {
94                  // If the vertex hasn't been visited, insert it in the
95                  if(visited.find(e.dest) == visited.end()) {
96                      stack.push(e.dest);
97                  }
98              }
99          }
100     }
101     return visit_order;
102 }
103
104 template <typename T>
105 auto create_reference_graph() {
106     Graph<T> G(9);
107     std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
108     edges[1] = {{2, 0}, {5, 0}};
109     edges[2] = {{1, 0}, {5, 0}, {4, 0}};
110     edges[3] = {{4, 0}, {7, 0}};
111     edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
112     edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
113     edges[6] = {{4, 0}, {7, 0}, {8, 0}};
114     edges[7] = {{3, 0}, {6, 0}};
115     edges[8] = {{4, 0}, {5, 0}, {6, 0}};
116     for (auto &i : edges)
117         for (auto &j : i.second)
118             G.add_edge(Edge<T>{i.first, j.first, j.second});
```

```
119     return G;
120 }
121
122 template <typename T>
123 void test_DFS() {
124     // Create an instance of and print the graph
125     auto G = create_reference_graph<unsigned>();
126     std::cout << G << std::endl;
127     // Run DFS starting from vertex ID 1 and print the order
128     // in which vertices are visited.
129     std::cout << "DFS Order of vertices: " << std::endl;
130     auto dfs_visit_order = depth_first_search(G, 1);
131     for (auto v : dfs_visit_order)
132         std::cout << v << std::endl;
133 }
134
135 int main() {
136     using T = unsigned;
137     test_DFS<T>();
138     return 0;
139 }
```

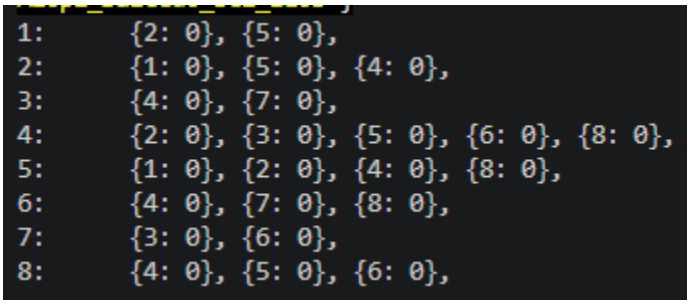
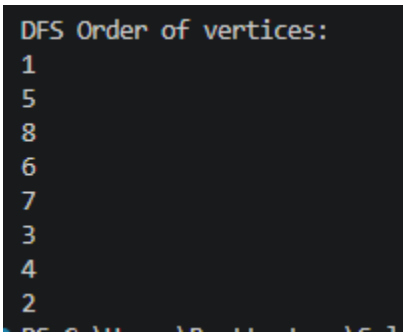
Output	 
Analysis	This code is an example of a Depth-First Search. It uses backtracking and stack whenever it reaches a dead end and pops itself (recursion) to go to another branch. It pushes to vertex 1, then it repeatedly pops the top vertex, it marks it as visited and therefore goes to another branch.

Table 10.2 ILO B - Depth First Search

Code

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
    for (auto i = 1; i < G.vertices(); i++) {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
```

```
32
33          os << std::endl;
34      }
35      return os;
36  }
37
38  template <typename T>
39  class Graph {
40  public:
41      Graph(size_t N) : V(N) {}
42      auto vertices() const {
43          return V;
44      }
45      auto &edges() const {
46          return edge_list;
47      }
48      void add_edge(Edge<T> &&e) {
49          if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V)
50              edge_list.emplace_back(e);
51          else
52              std::cerr << "Vertex out of bounds" << std::endl;
53      }
54      auto outgoing_edges(size_t v) const {
55          std::vector<Edge<T>> edges_from_v;
56          for (auto &e : edge_list) {
57              if (e.src == v) {
58                  edges_from_v.emplace_back(e);
59              }
60          }
```

```
61          return edges_from_v;
62      }
63
64      template <typename U>
65      friend std::ostream &operator<<(std::ostream &os,
66          const Graph<U> &G);
67
68  private:
69      size_t V;
70      std::vector<Edge<T>> edge_list;
71  };
72
73  template <typename T>
74  auto create_reference_graph() {
75      Graph<T> G(9);
76      std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
77      edges[1] = {{2, 2}, {5, 3}};
78      edges[2] = {{1, 2}, {5, 5}, {4, 1}};
79      edges[3] = {{4, 2}, {7, 3}};
80      edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
81      edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
82      edges[6] = {{4, 4}, {7, 4}, {8, 1}};
83      edges[7] = {{3, 3}, {6, 4}};
84      edges[8] = {{4, 5}, {5, 3}, {6, 1}};
85      for (auto &i : edges)
86          for (auto &j : i.second)
87              G.add_edge(Edge<T>{i.first, j.first, j.second});
```

```
    return G;
}

template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest) {
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1); // Assume that BFS always starts from vertex ID 1
    while (!queue.empty()) {
        auto current_vertex = queue.front();
        queue.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}

template <typename T>
void test_BFS() {
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

int main() {
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

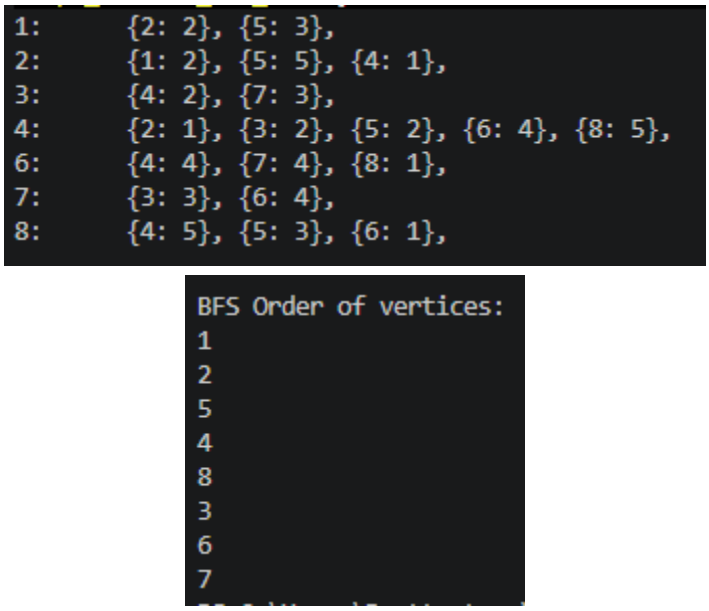
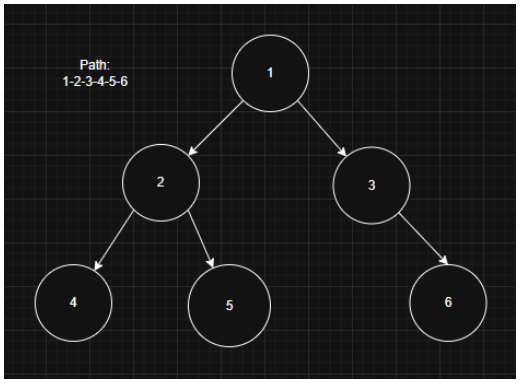
Output	
Analysis	This code is an example of Breadth-First Search. It explores all vertices at the current depth level before moving to the next level. It uses a queue for exploration. It enqueues first in vertex 1, then it loops to dequeue in the vortex, then it marks it as visited. After that it enqueues all its neighbors.

Table 10.3 ILO B - Breadth-First Search

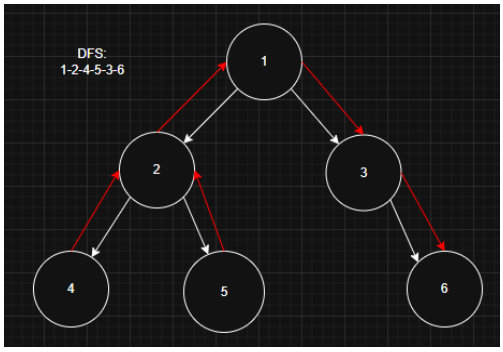
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Questions	Answers
1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.	The most suitable algorithm in my opinion is DFS or Depth-First Search. This is because DFS uses backtracks and can explore to another branch if it meets a dead end if it doesn't find it.
2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.	It is similar to the tree graph from the previous modules. Like pre-order, in-order and post-order.

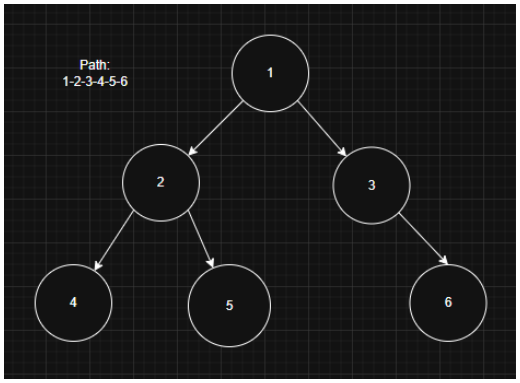
Path:
1→2→3→4→5→6



DFS Path:
1→2→4→5→3→6



Pre-Order:
1→2→4→5→3→6
In-Order:
4→2→5→1→3→6
Post-Order:
4→5→2→6→3→1



3. In the performed code, what data structure is used to implement the Breadth First Search?	It can't since Breadth-First Search uses queue while Depth-First Search uses stack.
4. How many times can a node be visited in the BFS?	Only once since it already visited and proceeded to go to another branch.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Brave Leo**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code analysis, questions**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used Brave Leo for code analysis and for the questions in the supplementary but all sentences made are mine only.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.